

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO 4: Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

Assessment Tool : Class Test

Weightage: 10%

QUESTIONS:

Total Marks: 20

1. Explain the fundamental concepts used in web development by defining the following terms with suitable explanations: XML, JSP, MVC architecture, XSLT, XML Schema, JSTL, PHP, Database Connectivity, and Parser. Your answer should clearly describe the purpose and role of each concept in web applications.
2. Explain how the above technologies are interconnected in building a web application. Describe how XML is structured and validated, how JSP and PHP are used for server-side processing, how MVC architecture organizes the application, and how database connectivity and parsers contribute to data processing and communication.

Code of Conduct:

I A.Lakshmi(192373066) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Lakshmi Ankala

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	25%	Clearly explains all concepts with complete accuracy and depth	Minor gaps in understanding	Basic understanding with some errors	Incorrect or very limited understanding
Content Coverage	25%	Covers all required topics comprehensively	Covers most topics with minor omissions	Covers some topics only	Very few topics covered
Technical Accuracy	20%	All explanations are technically correct and precise	Minor technical errors	Some incorrect or vague explanations	Major technical errors
Integration of Concepts	15%	Effectively connects and relates concepts logically	Some connections made with minor gaps	Limited connections between concepts	No clear relationship between concepts
Clarity & Presentation	15%	Well-structured, clear, and easy to understand	Mostly clear with minor issues	Somewhat unclear or poorly organized	Poorly structured and difficult to understand

1. XML – Extensible Markup Language

Definition

XML (Extensible Markup Language) is a markup language used to store, organize, and transport structured data.

Unlike HTML, XML is not mainly concerned with how data looks on a webpage. Instead, XML describes what the data represents.

Example

```
<student>
  <id>101</id>
  <name>Arun</name>
  <department>CSE</department>
  <mark>85</mark>
</student>
```

Here:

- <student> represents a student.
- <id> represents the student's ID.
- <name> represents the student's name.
- <department> represents the department.
- <mark> represents the student's mark.

The developer can create meaningful tags according to the application's requirements.

Main characteristics

- Self-descriptive
- Hierarchical
- Platform independent
- Human-readable
- Machine-readable
- Separates data from presentation
- Useful for data exchange

Role in web applications

XML can be used to:

- Exchange data between applications
- Store configuration information
- Transfer structured data
- Represent hierarchical information
- Provide data to transformation technologies such as XSLT

Simple example

Student Application

```
|
v
XML
|
+---- Student details
+---- Course details
+---- Marks
```

2. JSP – JavaServer Pages

Definition

JSP (JavaServer Pages) is a Java-based server-side technology used to create dynamic web pages.

With JSP, a web server processes the request and generates HTML that is sent to the client's browser.

Basic flow

Browser

|

| HTTP Request

v

Web Server

|

v

JSP

|

v

Java/Backend Processing

|

v

HTML Response

|

v

Browser

Example

A JSP page can display a username dynamically:

```
<h1>Welcome, ${userName}</h1>
```

The value of userName can be supplied by the backend.

Role in web applications

JSP can be used for:

- Dynamic HTML generation
- Displaying database information
- Processing user-related data
- Creating web forms
- Working with Java-based backend components

In a properly designed application, JSP is generally used mainly for the View layer rather than putting large amounts of business logic directly inside JSP.

3. MVC Architecture

Definition

MVC (Model–View–Controller) is an architectural pattern used to organize a web application into three separate components.

Three components

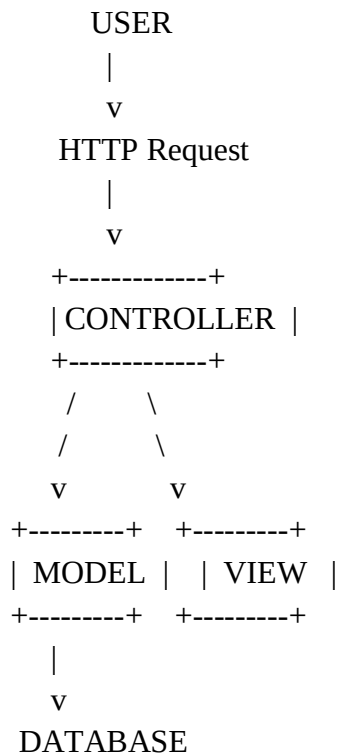
Component Responsibility

Model Data and business logic

View User interface/presentation

Controller Handles requests and coordinates Model and View

Architecture



Model

The Model manages:

- Application data
- Business rules
- Database interaction
- Data processing

View

The View is responsible for what the user sees.

Examples:

- HTML
- JSP
- Templates
- Web page components

Controller

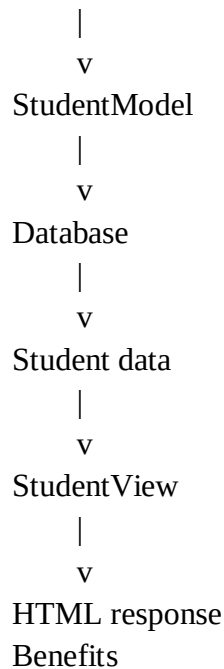
The Controller receives user requests and decides what should happen next.

For example:

User requests /students



StudentController



- Separation of concerns
- Easier maintenance
- Easier testing
- Better teamwork
- Easier modification of UI
- Better organization of large applications

4. XSLT – Extensible Stylesheet Language Transformations

Definition

XSLT is a language used to transform XML documents into other formats, particularly HTML, XML, and text.

The important distinction is:

XML contains the data, while XSLT contains the rules for transforming that data.

Example

Suppose XML contains:

```
<student>
  <name>Priya</name>
  <mark>90</mark>
</student>
```

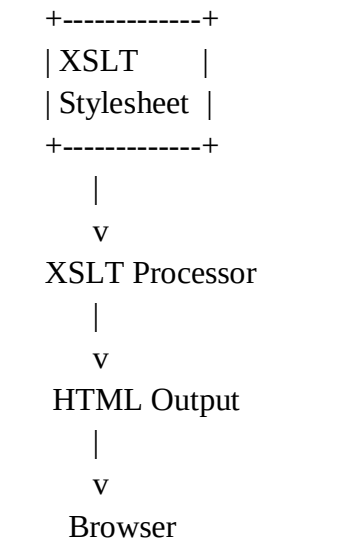
XSLT can transform it into:

```
<h2>Student Details</h2>
<p>Name: Priya</p>
<p>Mark: 90</p>
```

Flow

XML Data





Important XSLT concepts

XSLT commonly uses:

- Templates
- XPath
- Conditions
- Loops
- Sorting
- Filtering
- Node selection

Where XSLT is useful

XSLT is particularly useful for:

- XML reports
- Enterprise XML systems
- XML-to-HTML conversion
- Data publishing
- Legacy XML-based applications

5. XML Schema – XSD

Definition

XML Schema, commonly called XSD (XML Schema Definition), defines the rules and structure that an XML document must follow.

In simple words:

XML contains the data; XML Schema defines the rules for that data.

Example XML

```
<student>
  <name>Arun</name>
  <age>21</age>
  <mark>85</mark>
</student>
```

An XML Schema could specify that:

name → String

age → Integer

mark → Integer

mark → 0–100

If the XML contains:

<mark>ABC</mark>

the schema validation can identify that the value does not satisfy the expected data type.

What XSD can define

- Elements
- Attributes
- Data types
- Required fields
- Optional fields
- Minimum/maximum values
- Number of occurrences
- Patterns
- Relationships between elements

Validation flow

XML Document

|

v

XML Parser

|

v

XSD Schema

|

v

Validation

/ \

Invalid Valid

|

|

v

v

Error Application

Importance

XML Schema improves:

- Data consistency
- Data quality
- Reliability
- Interoperability
- Validation of exchanged XML data

6. JSTL – JavaServer Pages Standard Tag Library

Definition

JSTL (JSP Standard Tag Library) is a collection of standard tags that can be used inside JSP pages to perform common tasks without writing large amounts of Java code directly in the JSP.

Instead of writing Java scriptlets everywhere, developers can use tags.

Example

A conditional operation can be represented conceptually as:

```
<c:if test="${student.mark >= 50}">
```

Pass

```
</c:if>
```

A loop can be represented as:

```
<c:forEach var="student" items="${students}">
```

 \${student.name}

```
</c:forEach>
```

Major JSTL categories

Category	Purpose
Core	Conditions, loops, variables
Formatting	Number/date formatting
SQL	Database-related operations
XML	XML processing
Functions	String and utility functions

Why JSTL is useful

Without JSTL:

JSP

|

+-- HTML

+-- Java code

+-- Java code

+-- Java code

With JSTL:

JSP

|

+-- HTML

+-- JSTL tags

+-- Expression Language

This makes JSP pages cleaner and easier to maintain.

7. PHP – Hypertext Preprocessor

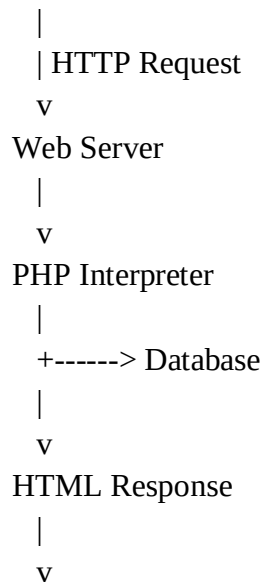
Definition

PHP is a server-side scripting language commonly used to develop dynamic web applications.

PHP executes on the server, generates a response, and sends the result to the browser.

Basic flow

Browser



Browser

Example

```
<?php
$name = "Arun";
echo "Welcome " . $name;
?>
```

The server processes this code and sends the resulting output to the browser.

PHP can be used for

- Login systems
- Registration forms
- E-commerce applications
- Content management systems
- Database-driven websites
- REST APIs
- Session management

Popular PHP frameworks include:

- Laravel
- Symfony
- CodeIgniter

8. Database Connectivity

Definition

Database connectivity is the mechanism through which a web application communicates with a database to store, retrieve, update, and delete data.

For example, a student application might communicate with:

MySQL

PostgreSQL

Oracle

SQL Server

Basic flow

Web Application

|

v

Database Connectivity Layer

|

v

SQL Query

|

v

Database

|

v

Result

|

v

Application

In Java

Java applications commonly use JDBC (Java Database Connectivity).

Typical process:

Load Driver

↓

Create Connection

↓

Create Statement

↓

Execute Query

↓

Process ResultSet

↓

Close Resources

In PHP

PHP applications can use technologies such as:

- PDO
- MySQLi
- Framework database/ORM layers

Example operation

A user asks to view product 101:

User

|

v

Controller

|

v

Database Layer

|
v
SELECT * FROM products WHERE id = 101

|
v
Database

|
v
Product Data

|
v
View

|
v
User

Database connectivity is therefore the bridge between the application layer and persistent data.

9. Parser

Definition

A parser is software that reads input according to a defined syntax or structure and converts it into a form that a program can understand and process.

In web development, parsers are used for different types of data, including:

- XML
- HTML
- JSON
- Programming languages

XML Parser

An XML parser reads an XML document and checks its structure.

Example:

```
<student>
  <name>Arun</name>
  <mark>85</mark>
</student>
```

The parser identifies:

```
student
├── name
│   └── Arun
└── mark
    └── 85
```

Common XML parsing approaches

DOM Parser

Loads the XML document into a tree structure.

XML

|

v

DOM Tree

|

v

Application

SAX Parser

Reads XML sequentially and generates events.

XML

|

v

SAX Parser

|

+--> startElement()

+--> characters()

+--> endElement()

Why parsers are important

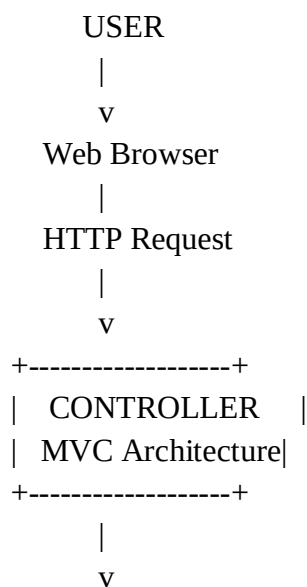
A parser:

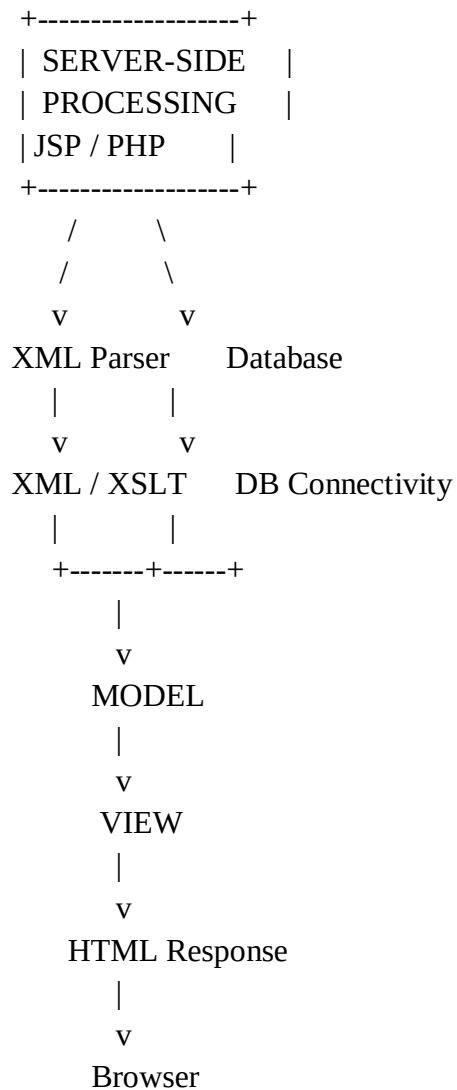
- Reads structured data
- Checks syntax
- Identifies elements
- Extracts values
- Makes data available to applications
- Helps detect malformed input

2. How These Technologies Are Interconnected

The important thing to understand is that these technologies do not necessarily perform the same job. They form different layers of a web application.

A simplified architecture is:





Let's understand the connection step by step.

Step 1: User Sends a Request

The process starts when a user interacts with a web application.

For example:

User clicks:

"View Product"

The browser sends an HTTP request:

GET /product/101

Step 2: Controller Receives the Request

In an MVC architecture, the request is handled by the Controller.

For example:

ProductController

The Controller determines what operation needs to be performed.

Request

|
v

ProductController

```
|
+--> Validate request
|
+--> Call Model
```

This is where MVC becomes important.

Step 3: Model Processes the Data

The Controller communicates with the Model.

The Model is responsible for application data and business logic.

For example:

Controller

```
|
v
```

Product Model

```
|
+---- Database
|
+---- XML
|
+---- External API
```

Step 4: Database Connectivity Retrieves Data

If product information is stored in a database, the Model uses a database connectivity mechanism.

For Java:

Model

```
|
v
```

JDBC / JPA / Hibernate

```
|
v
```

Database

For PHP:

Model

```
|
v
```

PDO / MySQLi / ORM

```
|
v
```

Database

The database returns the requested information.

Example:

ID 101

Name Laptop
Price ₹55,000
Stock 25

Step 5: XML Can Represent Structured Data

Sometimes the application receives or exchanges information in XML format.

For example:

```
<product>  
  <id>101</id>  
  <name>Laptop</name>  
  <price>55000</price>  
</product>
```

This XML provides a structured representation of the data.

Step 6: Parser Reads XML

The application needs an XML parser to understand the XML structure.

XML

|

v

Parser

|

v

XML Tree / Events

|

v

Application

For smaller documents where random access is useful, DOM can be used.

For very large documents, SAX can be more memory-efficient.

Step 7: XML Schema Validates the XML

Before XML data is processed, it can be checked against an XML Schema.

Incoming XML

|

v

XML Parser

|

v

XML Schema

|

v

Valid?

/ \

NO YES

|

|

v v

Error Process

For example, the schema may require:

product.id → integer

product.name → string

product.price → decimal

This prevents structurally invalid XML from entering further processing.

Step 8: XSLT Can Transform XML

If the XML data needs to be converted into a readable report, XSLT can transform it.

XML

|

v

XSLT Stylesheet

|

v

XSLT Processor

|

v

HTML

For example:

<product>

 <name>Laptop</name>

 <price>55000</price>

</product>

can be transformed into:

<h2>Laptop</h2>

<p>Price: ₹55,000</p>

So XSLT is mainly concerned with transformation and presentation of XML data.

Step 9: JSP Can Generate the View

In a Java web application, JSP can act as the View.

The Controller provides the required data to the JSP.

Controller

|

v

Data

|

v

JSP

|

v

HTML

JSTL can be used inside JSP to simplify common presentation logic.

For example:

```
<c:forEach var="product" items="${products}">
    ${product.name}
</c:forEach>
```

So the relationship is:

JSP

|

+---- HTML

|

+---- JSTL

|

+---- Expression Language

|

+---- Data from Model

Step 10: PHP Can Perform Server-Side Processing

PHP can perform a similar server-side role in a PHP-based application.

For example:

Browser

|

v

PHP Application

|

+---- Controller

|

+---- Model

|

+---- Database

|

v

HTML Response

Modern PHP frameworks such as Laravel and Symfony provide MVC structures to organize this processing.

Complete Interconnection

The entire concept can be represented as:

USER

|

v

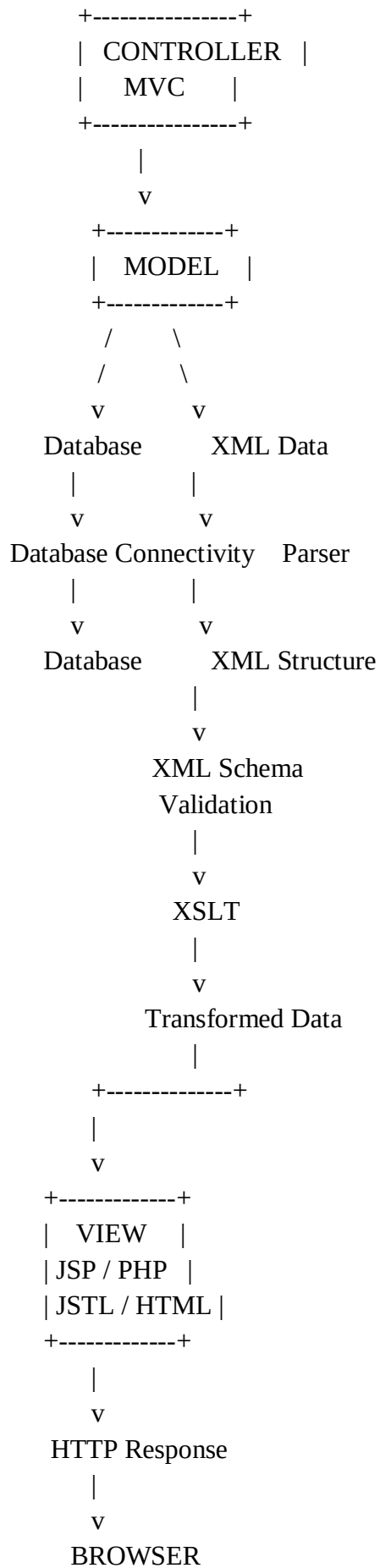
BROWSER

|

HTTP Request

|

v



|
v
USER

How the Technologies Work Together

Technology	Main Role	Connects With
XML	Stores/exchanges structured data	Parser, XSD, XSLT
XML Schema	Validates XML structure and data types	XML, Parser
Parser	Reads and interprets structured data	XML, application
XSLT	Transforms XML into HTML/XML/text	XML, XSLT processor
JSP	Generates dynamic Java web views	MVC, Servlet, JSTL
JSTL	Simplifies common JSP view logic	JSP
PHP	Server-side application processing	Database, MVC, HTML
MVC	Organizes application components	Controller, Model, View
Database Connectivity	Connects application to database	Model, database

Example: Student Management Web Application

Let's put everything together using a realistic example.

Suppose a university has a Student Result Management System.

1. User

The student enters:

<https://university.com/result/101>

2. Controller

The Controller receives the request.

ResultController

3. Model

The Model asks the database for student information.

ResultModel

|

v

Database Connectivity

|

v

Database

4. Database

The database returns:

ID: 101

Name: Arun

Department: CSE

Mark: 85

5. XML

Suppose the data also needs to be exchanged as XML:

```
<student>
  <id>101</id>
  <name>Arun</name>
  <department>CSE</department>
  <mark>85</mark>
</student>
```

6. Parser

The XML parser reads the XML structure.

7. XML Schema

The XML is validated:

```
id      → integer
name    → string
department → string
mark    → integer
```

8. XSLT

If a printable XML report is required:

XML

|

v

XSLT

|

v

HTML Report

9. JSP

The Java application can use JSP as its View.

JSTL can help display multiple student records.

10. Response

Finally:

Controller

|

v

JSP View

|

v

HTML

|

v

Browser

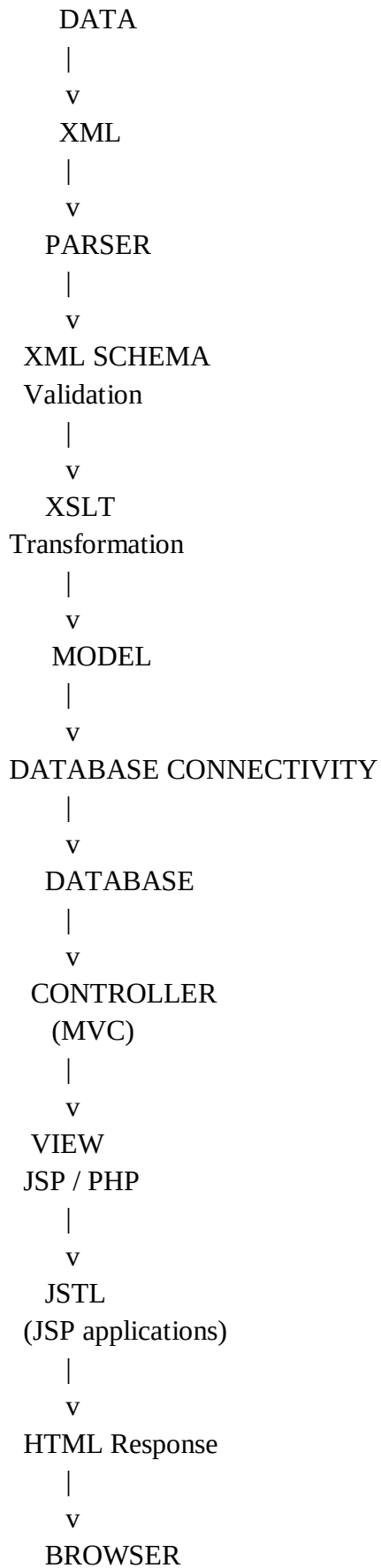
|

v

Student

Final Summary

The easiest way to remember all these concepts is to think of a web application as a pipeline:



However, this is a conceptual relationship, not a rule that every application must use every technology. A modern application might use JSON instead of XML, React instead of JSP, and an ORM instead of direct JDBC.

The key idea is:

- XML → represents structured data.
- XML Schema → checks whether XML follows defined rules.
- Parser → reads structured data so the application can process it.
- XSLT → transforms XML into another representation.
- Database Connectivity → allows the application to communicate with a database.
- Model → manages application data and business logic.
- Controller → receives and coordinates requests.
- JSP/PHP → can perform server-side processing and generate responses.
- JSTL → provides standard tags that make JSP presentation logic cleaner.
- MVC → organizes all these application responsibilities into manageable components.

Together, these concepts form the foundation for understanding how data moves from a user request, through server-side processing and data sources, and finally back to the user's browser as a usable web page.